MODULE

08

Pythonic code

Write Python the way experienced developers do: shorter, clearer, and fewer bugs.

YOU WILL LEARN

Comprehensions	01_comprehensions.py
Unpacking	02_unpacking.py
Looping like a pro	03_looping_like_a_pro.py
Truthiness & walrus	04_truthiness_and_walrus.py
Strings like a pro	05_strings_like_a_pro.py
Pattern matching	07_pattern_matching.py
Regular expressions	08_regex.py
Interview Q&A	INTERVIEW.md

WHY PYTHONIC CODE

What does ”Pythonic” mean?

THE PROBLEM

Two programs can give the same result. One is **12 lines** of index juggling; the other is **3 clear lines**.

In code reviews and interviews, people judge you on the second kind.

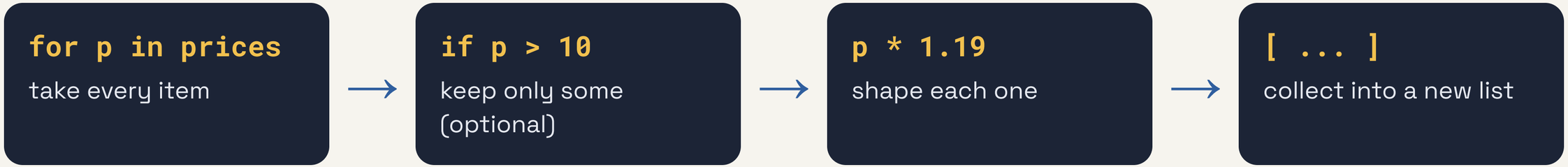
THE IDEA

Pythonic = using the language’s own tools (comprehensions, unpacking, enumerate, zip) so the code reads like English.

PICTURE IT COMPREHENSIONS

A loop squeezed into one line

🏭 Think of a **factory line**: items come in (**for**), a quality check throws some out (**if**), and each good item is shaped (**expression**) before it goes in the box.



Read it in this order: **[shape for item in items if check]**

TRY IT COMPREHENSIONS

Lists, dicts and sets in one line

```
prices = [5, 12, 30, 8]

# the long way
with_vat = []
for p in prices:
    if p > 10:
        with_vat.append(round(p * 1.19, 2))
print(with_vat)

# the Pythonic way
print([round(p * 1.19, 2) for p in prices if p > 10])

names = ["vishal", "li", "sam"]
print({n: len(n) for n in names})        # dict
print({len(n) for n in names})           # set: no duplicates
```

OUTPUT

```
[14.28, 35.7]
[14.28, 35.7]
{'vishal': 6, 'li': 2, 'sam': 3}
{2, 3, 6}
```

1. Same result, **one line**, and it's faster.
2. **{k: v for ...}** builds a dict.
3. **{x for ...}** builds a set; **(x for ...)** is a lazy generator (Module 11).

COMMON MISTAKE COMPREHENSIONS

When NOT to use a comprehension

✗ WRONG

```
[print(u) for u in users]          # side effect

grid = [[c for c in row if c]
        for row in data if row
        if len(row) > 2]           # unreadable
```

A comprehension **builds a list**. Using it only to print or save throws that list away. Deep nesting is hard to read.

✓ RIGHT

```
for u in users:                    # a plain loop
    print(u)

def clean(row):                    # name the step
    return [c for c in row if c]

grid = [clean(r) for r in data if len(r) > 2]
```

Rule: if you need a **result**, use a comprehension. If you need an **action**, or it doesn't fit in one line, write a loop.

TRY IT UNPACKING

Take values out of a sequence by position

```
a, b = 1, 2
a, b = b, a                    # swap: no temp variable
print(a, b)

first, *middle, last = [10, 20, 30, 40, 50]
print(first, middle, last)

name, (city, zip_code) = "Vishal", ("Augsburg", "86150")
print(name, city, zip_code)

defaults = {"theme": "light", "lang": "en"}
user = {"lang": "de"}
print({**defaults, **user})    # merge: right side wins
```

OUTPUT

```
2 1
10 [20, 30, 40] 50
Vishal Augsburg 86150
{'theme': 'light', 'lang': 'de'}
```

1. The right side is built **first**, then assigned: that's why swapping works.
2. ***name** collects "the rest" into a list.
3. **{**a, **b}** merges dicts (also **a | b** in 3.9+).

COMMON MISTAKE LOOPING LIKE A PRO

Stop writing range(len(...))

✗ WRONG

```
names = ["vishal", "li"]
scores = [91, 78]
for i in range(len(names)):
    print(i, names[i], scores[i])
```

Index juggling is noisy and breaks easily (IndexError when the lists have different lengths).

✓ RIGHT

```
for i, (name, score) in enumerate(zip(names, scores), start=1):
    print(i, name, score)
```

enumerate gives the counter, **zip** walks several lists together. No indexes, no IndexError.

TRY IT LOOPING LIKE A PRO

Sort, search and check without loops

```
users = [{"name": "vishal", "age": 31}, {"name": "li", "age": 25},
         {"name": "sam", "age": 40}]

by_age = sorted(users, key=lambda u: u["age"])
print([u["name"] for u in by_age])

print(max(users, key=lambda u: u["age"])["name"])
print(any(u["age"] > 35 for u in users))   # at least one?
print(all(u["age"] > 18 for u in users))   # every one?
print(sum(u["age"] for u in users) / len(users))
```

OUTPUT

```
['li', 'vishal', 'sam']
sam
True
True
32.0
```

1. **key=** says **what** to compare.
2. **any** / **all** stop as soon as the answer is known.
3. **sorted()** returns a new list; **.sort()** changes the list in place.

TRY IT TRUTHINESS

Every value is either truthy or falsy

```
for value in [0, "", [], {}, None, "0", [0], -1]:
    print(f"{value!r:6} -> {bool(value)}")
```

OUTPUT

```
0      -> False
''     -> False
[]     -> False
{}     -> False
None   -> False
'0'    -> True
[0]    -> True
-1     -> True
```

1. **Falsy**: zero, empty containers, **None**, **False**.
2. Everything else is **truthy**, even **"0"** and **[0]**.
3. So **if items:** means "if the list is not empty".

COMMON MISTAKE TRUTHINESS

When ”if x” is a bug

✗ WRONG

```
def apply_discount(price, discount=None):
    if not discount:                # 0 is falsy!
        discount = 0.10             # default 10%
    return price * (1 - discount)

print(apply_discount(100, 0))     # 90.0  ???
```

A real **0** (no discount) is falsy, so the code silently replaces it with 10%. Customers get a discount they shouldn’t.

✓ RIGHT

```
def apply_discount(price, discount=None):
    if discount is None:          # only "missing"
        discount = 0.10
    return price * (1 - discount)

print(apply_discount(100, 0))     # 100
```

Test for **None** with **is None** when 0, ”” or [] are valid values.

TRY IT WALRUS :=

Assign and test in one step

```
import re

lines = ["GET /home 200", "GET /admin 403", "POST /login 500"]
for line in lines:
    if (m := re.search(r"(\d{3})$", line)) and m.group(1) >= "400":
        print("problem:", line)

data = [3, 8, 1, 9]
if (n := len(data)) > 3:
    print(f"list too long ({n} items)")
```

OUTPUT

```
problem: GET /admin 403
problem: POST /login 500
list too long (4 items)
```

1. **:=** stores the value **and** returns it.
2. Avoids computing the same thing twice.
3. Use it in **if** and **while**; don't sprinkle it everywhere.

TRY IT STRINGS LIKE A PRO

f-strings can format anything

```
name, price, ratio, n = "Lamp", 1234.5, 0.4567, 7

print(f"{name:<8}|{price:>12,.2f}|")   # align + thousands
print(f"{ratio:.1%}")                   # percentage
print(f"{n:03d}")                       # zero padding
print(f"{price=}")                      # debug print (3.8+)

words = ["fast", "clear", "pythonic"]
print(", ".join(words))
print("a,b,,c".split(","))
```

OUTPUT

```
Lamp    |    1,234.50|
45.7%
007
price=1234.5
fast, clear, pythonic
['a', 'b', '', 'c']
```

1. **:<8** left, **:>12** right, **,.2f** thousands + 2 decimals.
2. **{x=}** prints the name and the value: great for debugging.
3. **join** glues a list into one string; **split** does the opposite.

COMMON MISTAKE STRINGS LIKE A PRO

Building a string in a loop

✗ WRONG

```
report = ""
for row in rows:
    report += row + "\n"    # copies the whole
                             # string every time
```

Strings are **immutable**: every **+=** creates a new, longer string. With 100,000 rows that's very slow (quadratic).

✓ RIGHT

```
report = "\n".join(rows)

# or collect parts first
parts = [f"{r.id}: {r.total}" for r in rows]
report = "\n".join(parts)
```

Collect pieces in a **list**, then **join** once. Linear time.

TRY IT PATTERN MATCHING (3.10+)

match / case: branch on the SHAPE of data

```
def handle(event):
    match event:
        case {"type": "order", "items": [first, *rest]}:
            return f"order: {first} (+{len(rest)} more)"
        case {"type": "refund", "amount": float(a)} if a > 0:
            return f"refund {a:.2f} EUR"
        case {"type": "ping"}:
            return "pong"
        case _:
            return "unknown event"

print(handle({"type": "order", "items": ["lamp", "cable", "plug"]}))
print(handle({"type": "refund", "amount": 19.9}))
print(handle({"type": "ping", "extra": 1}))
print(handle({"oops": True}))
```

OUTPUT

```
order: lamp (+2 more)
refund 19.90 EUR
pong
unknown event
```

1. Matches **dicts** (extra keys allowed), **lists**, **types** and **classes**.
2. A name in a pattern **captures** the value.
3. **if** after a pattern is a **guard**; **_** matches anything.

TRY IT REGULAR EXPRESSIONS

Find patterns in text

```
import re

log = "2026-11-08 12:01 ERROR user=vishal ip=10.0.0.7 took=532ms"

m = re.search(r"(?P<level>[A-Z]+) user=(?P<user>\w+)", log)
print(m["level"], m["user"])

print(re.findall(r"\d+ms", log))
print(re.sub(r"ip=\S+", "ip=***", log))

EMAIL = re.compile(r"^[\w.+-]+@[\w-]+\.[\w.]+$")
print(bool(EMAIL.match("vishal@example.com")), bool(EMAIL.match("nope@")))
```

OUTPUT

```
ERROR vishal
['532ms']
2026-11-08 12:01 ERROR user=vishal
ip=*** took=532ms
True False
```

1. **(?P<name>...)** = a named group you read with **m["name"]**.
2. **findall** = every match; **sub** = search and replace.
3. Always use raw strings: **r"..."**.

IN REAL JOBS PYTHONIC CODE

Where you'll use this every day

Turn DB rows into API JSON

```
return [{"id": o.id, "total": o.total}
        for o in orders if o.paid]
```

Merge config and defaults

```
settings = {**DEFAULTS, **env_settings}
```

Mask secrets in logs

```
re.sub(r"token=\S+", "token=***", line)
```

Route webhook events

```
match payload:
    case {"event": "payment.succeeded"}: ...
```

INTERVIEW SAY THIS IN THE INTERVIEW

“What’s the difference between is and ==?”

- **==** compares **values**: do these two objects contain the same data?
- **is** compares **identity**: are these the **same object** in memory (same id)?
- Use **is** only for singletons: **None**, **True**, **False**.
- Small ints and short strings may be cached, so **is** can look right by accident. Never rely on it.

```
a = [1, 2]
b = [1, 2]
print(a == b)    # True   same values
print(a is b)    # False  two objects
print(a is a)    # True
x = None
print(x is None) # the right way
```

Follow-up they ask next: ”What is a list comprehension vs a generator expression?” ([] builds the whole list now; () makes values lazily, one by one).

INTERVIEW SAY THIS IN THE INTERVIEW

“How do you make code more Pythonic?”

- Use **comprehensions** instead of append loops, when building a result.
- **enumerate** and **zip** instead of **range(len(...))**.
- **Unpacking**, **f-strings**, **join**, and **sorted(key=...)**.
- **EAFP**: try the operation and catch the error, instead of checking everything first.
- Follow **PEP 8** and let **ruff** point out the rest (Module 15).

> **Follow-up they ask next:** ”What does EAFP vs LBYL mean?” (Easier to Ask Forgiveness than Permission vs Look Before You Leap: Python prefers try/except).

RECAP REMEMBER THESE

Module 8 in 6 lines

1. **[shape for x in xs if check]**, but use a loop for side effects.
2. **a, b = b, a**; **first, *rest = xs**; **{**a, **b}**.
3. **enumerate**, **zip**, **sorted(key=)**, **any**/**all**: no index juggling.
4. Empty and zero are falsy; use **is None** when 0 is valid.
5. f-strings format everything; build big strings with **join**.
6. **match/case** for shapes, **re** with named groups for text.

PRACTICE NOW

Do exercises/ex08_refactor_to_pythonic.py: turn clumsy code into Pythonic code.